



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа 9
по курсу «Методы Оптимизация»
Метод Ньютона.

Студент группы ИУ9-81Б Еделькин Г. Ю.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать Метод Ньютона. Сравнить с модификацией, когда обратный гессиан вычисляется один раз.

2 Результаты

было получено следующее 1

Листинг 1: реализация приложения

```
1
2 using LinearAlgebra
3 using Plots
4 using Printf
5 plotly()
6
7 # =====
8 # 1) Rosenbrock (2D)
9 # =====
10 f(x) = (1 - x[1])^2 + 100*(x[2] - x[1]^2)^2
11
12 function grad_f(x)
13     gx = -2*(1 - x[1]) - 400*x[1]*(x[2] - x[1]^2)
14     gy = 200*(x[2] - x[1]^2)
15     return [gx, gy]
16 end
17
18 function hess_f(x)
19     x1, x2 = x[1], x[2]
20     h11 = 2 - 400*(x2 - 3*x1^2)
21     h12 = -400*x1
22     h22 = 200.0
23     return [h11 h12;
24             h12 h22]
25 end
26
27 # =====
28 # 2) Backtracking Armijo ( )
29 # =====
30 function backtracking_armijo(f, x, p, g; 0 = 1.0, = 0.5, c = 1e-4, maxit
    = 30)
31     fx = f(x)
32     = 0
33     for _ in 1:maxit
34         x_try = x .+ .* p
35         if f(x_try) <= fx + c* *dot(g, p)
36             return
37         end
38         *=
39     end
40     return
```

```

41 end
42
43 # =====
44 # 3) Newton (H(xk)
45 # =====
46 function newton_method(f, grad_f, hess_f; x0, eps=1e-6, maxit=80,
    use_linesearch=true)
47     x = copy(x0)
48     path = Vector{Vector{Float64}}{ }()
49     push!(path, copy(x))
50
51     for k in 1:maxit
52         g = grad_f(x)
53         if norm(g) < eps
54             return x, path, k
55         end
56
57         H = hess_f(x)
58         s = H \ g
59         p = -s
60
61         = use_linesearch ? backtracking_armijo(f, x, p, g) : 1.0
62         x = x .+      .* p
63         push!(path, copy(x))
64     end
65
66     return x, path, maxit
67 end
68
69 # =====
70 # 4) Fixed inverse Hessian:  $H(x_0)^{-1}$ 
71 # =====
72 function newton_fixed_hessian(f, grad_f, hess_f; x0, eps=1e-6, maxit
    =250, use_linesearch=true)
73     x = copy(x0)
74     path = Vector{Vector{Float64}}{ }()
75     push!(path, copy(x))
76
77     H0inv = inv(hess_f(x0))
78
79     for k in 1:maxit
80         g = grad_f(x)
81         if norm(g) < eps
82             return x, path, k
83         end
84

```

```

85         p = -(H0inv * g)
86         = use_linesearch ? backtracking_armijo(f, x, p, g) : 1.0
87         x = x .+      .* p
88         push!(path, copy(x))
89     end
90
91     return x, path, maxit
92 end
93
94 # =====
95 # 5) Utility:
96 # =====
97 function auto_window(paths; pad=0.25)
98     xs = Float64[]; ys = Float64[]
99     for path in paths
100         append!(xs, (p[1] for p in path))
101         append!(ys, (p[2] for p in path))
102     end
103     xmin, xmax = minimum(xs), maximum(xs)
104     ymin, ymax = minimum(ys), maximum(ys)
105     dx = max(1e-9, xmax - xmin)
106     dy = max(1e-9, ymax - ymin)
107     return (xmin - pad*dx, xmax + pad*dx), (ymin - pad*dy, ymax + pad*dy)
108 end
109
110 # =====
111 # 6) 2D contours + two paths
112 # =====
113 function plot_2d_two_paths(f, path1, path2; xr, yr, n=240)
114     xs = range(xr[1], xr[2], length=n)
115     ys = range(yr[1], yr[2], length=n)
116     Z = [f([x,y]) for y in ys, x in xs]
117
118     p = contour(xs, ys, Z;
119         levels=40, linewidth=1,
120         c=:grays,
121         xlabel="x1", ylabel="x2",
122         title="Rosenbrock: contours + trajectories",
123         colorbar_title="f(x1,x2)",
124         legend=:topright
125     )
126
127     # Newton
128     x1 = [q[1] for q in path1]; y1 = [q[2] for q in path1]
129     plot!(p, x1, y1; lw=3, c=:dodgerblue, label="Newton")

```

```

130     scatter!(p, [x1[1]], [y1[1]]; ms=7, mc=:dodgerblue, label=false)
131     scatter!(p, [x1[end]], [y1[end]]; ms=8, mc=:dodgerblue, marker=:x,
132     label=false)
133
134     # Fixed  $H^{-1}(x_0)$ 
135     x2 = [q[1] for q in path2]; y2 = [q[2] for q in path2]
136     plot!(p, x2, y2; lw=3, c=:orange, label="Fixed  $H^{-1}(x_0)$ ")
137     scatter!(p, [x2[1]], [y2[1]]; ms=7, mc=:orange, label=false)
138     scatter!(p, [x2[end]], [y2[end]]; ms=8, mc=:orange, marker=:x, label=
139     false)
140
141     return p
142 end
143
144 # =====
145 # 7) 3D: "flag-like" coloring (white-blue-red) without textures
146 # We emulate flag by mapping colors by Y coordinate using a custom
147 # gradient.
148 # =====
149
150 function russian_flag_colormap()
151     # white -> blue -> red
152     # (Plots.jl understands ColorGradient from these symbols/Colors)
153     return cgrad([:white, :royalblue, :red], [0.0, 0.5, 1.0])
154 end
155
156 function plot_3d_two_paths_flag(f, path1, path2; xr, yr, n=140)
157     xs = range(xr[1], xr[2], length=n)
158     ys = range(yr[1], yr[2], length=n)
159
160     # Z values (surface height)
161     Z = [f([x,y]) for y in ys, x in xs]
162
163     # "Color field": use Y normalized to [0,1] so the surface looks like
164     # horizontal bands
165     ymin, ymax = yr
166     C = [ (y - ymin) / (ymax - ymin + 1e-12) for y in ys, x in xs ]
167
168     p = surface(xs, ys, Z;
169         c = russian_flag_colormap(), # our custom "flag" palette
170         fill_z = C, # IMPORTANT: colors come from C,
171         not Z
172         alpha=0.88,
173         xlabel="x1", ylabel="x2", zlabel="f(x1,x2)",
174         title="Rosenbrock: 3D surface (flag-like coloring) +
175         trajectories",
176         legend=:topright

```

```

170     )
171
172     # Newton path on surface
173     x1 = [q[1] for q in path1]; y1 = [q[2] for q in path1]; z1 = [f(q)
174     for q in path1]
175     plot!(p, x1, y1, z1; lw=4, c=:black, label="Newton") # black
176
177     # Fixed Hessian path on surface
178     x2 = [q[1] for q in path2]; y2 = [q[2] for q in path2]; z2 = [f(q)
179     for q in path2]
180     plot!(p, x2, y2, z2; lw=4, c=:gold, label="Fixed  $H^{-1}(x_0)$ ")
181     return p
182 end
183 # =====
184 # 8) Run + timings + accuracy
185 # =====
186 x0 = [-1.2, -1.0]
187 eps = 1e-6
188
189 tN = @elapsed begin
190     xN, pathN, itN = newton_method(f, grad_f, hess_f; x0=x0, eps=eps,
191     maxit=1000, use_linesearch=true)
192 end
193 # need values (since @elapsed scoped)
194 xN, pathN, itN = newton_method(f, grad_f, hess_f; x0=x0, eps=eps, maxit
195     =1000, use_linesearch=true)
196
197 tF = @elapsed begin
198     xF, pathF, itF = newton_fixed_hessian(f, grad_f, hess_f; x0=x0, eps=
199     eps, maxit=200000, use_linesearch=true)
200 end
201 xF, pathF, itF = newton_fixed_hessian(f, grad_f, hess_f; x0=x0, eps=eps,
202     maxit=200000, use_linesearch=true)
203
204 # Accuracy metrics
205 fN, gN = f(xN), norm(grad_f(xN))
206 fF, gF = f(xF), norm(grad_f(xF))
207
208 println("==== Newton (recompute Hessian) =====")
209 @printf("iters: %d | time: %.6f s | x*: [%.10f, %.10f] | f(x*): %.10e |
210     ||grad||: %.3e\n",
211     itN, tN, xN[1], xN[2], fN, gN)
212

```

```

208 println("\n===== Fixed inverse Hessian  $H^{-1}(x_0)$  =====")
209 @printf("iters: %d | time: %.6f s | x*: [% .10f, % .10f] | f(x*): % .10e |
      || grad ||: % .3e\n",
210         itF, tF, xF[1], xF[2], fF, gF)
211
212 # Plots (2 graphs total)
213 xr, yr = auto_window([pathN, pathF]; pad=0.35)
214
215 p2d = plot_2d_two_paths(f, pathN, pathF; xr=xr, yr=yr)
216 display(p2d)
217
218 p3d = plot_3d_two_paths_flag(f, pathN, pathF; xr=xr, yr=yr)
219 display(p3d)

```

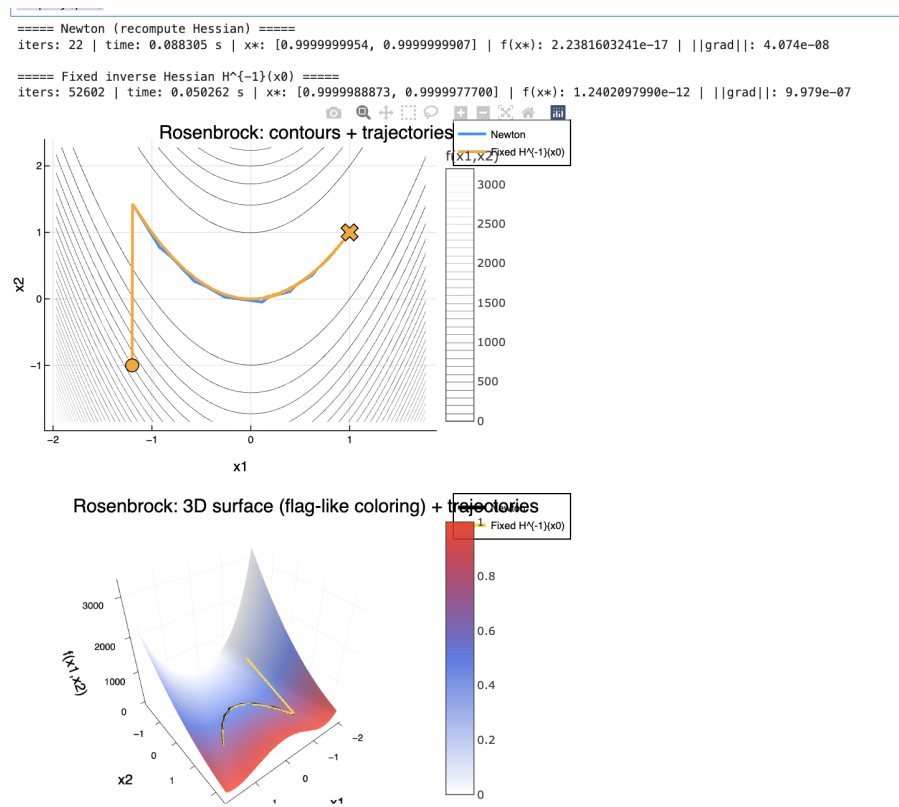


Рис. 1 — Результат для Резенброка

В результате получается следующее 1